

COLLABORATION STRUCTURE AND PERFORMANCE IN NEW SOFTWARE DEVELOPMENT: FINDINGS FROM THE STUDY OF OPEN SOURCE PROJECTS

JORGE A. COLAZO

*Business Administration Department
Trinity University, 1 Trinity Place, San Antonio, TX 78212
jcolazo@trinity.edu*

The state of software development performance is far from being exemplar, with a success rate well below that of other industries, and understanding how to improve these projects is not only substantive but urgent. Software development is at most times a collaborative effort, yet we do not understand clearly how different collaboration structures associate with development performance metrics. This paper empirically examines the association between collaboration patterns, project productivity and product quality through a field study of working software open source new product development teams, using archival data from electronic sources related to Open Source Software projects. Collaboration structures are measured using Social Network Analysis and in terms of their network density, network centralization and the level of boundary spanning activity of team members. Productivity and quality are measured using “hard”, code-based metrics. Results of regression analyses testing relevant hypotheses suggest that project managers need to strongly encourage internal collaboration, but be wary of allowing team members to participate in multiple projects. The breadth of skills within the team is a tactical asset that may increase the efficient frontier in the quality–productivity trade-off. Results also show that centralized structures associate with higher product quality and development productivity.

Keywords: Software development; collaboration; productivity; quality; social network analysis.

Introduction

Almost since its inception, the software industry has been in a state of turmoil with regards to the effective introduction of new products. By virtually any measure, the success of new software development has been disappointing. Reports in business periodicals have known that software innovation has a low success rate, close to a dismal four percent ([Business Week, 2005](#)).

Software development project failure has several facets. For instance, we do not fully understand how to sustain high project productivity in order to avoid missing project deadlines. We do not know how to consistently produce programs that are not riddled with defects that aggravate users and require frequent and disruptive product updates (Cusumano, 2004). Perhaps more importantly, problems in the software innovation process eventually translate into big monetary costs. For example, the new air traffic control system for the United States Federal Aviation Administration was reported to have “completely wasted” \$1.5 billion (out of \$2.6 billion spent) on various software development problems (U.S. House of Representatives, 2001). It is plausible to expect that the increasing complexity of software applications will only worsen this problem in the near future.

Project managers have tried to improve the software development process by applying a multitude of management models and techniques. From the waterfall model in the 1970s (Royce, 1970) to the more recent extreme programming methodology (Beck, 1999), progress made has not caught up with the exponential rate of increase in complexity of today’s applications (Gordon, 2003).

In the last decade, however, a new and intriguing software development paradigm has attracted the attention of businesses and academics: the Open Source Software (OSS) paradigm. OSS is software under licensing terms that make the complete source code (instructions coded in a programming language) of the product publicly available and allow the redistribution of modified versions. OSS has thus generated a particular innovation model, one in which the programs are developed collaboratively by dispersed teams of mostly volunteer individuals who tap into the publicly available source code and cooperate to develop the software, making intensive use of electronically mediated communication.

Well known examples of OSS are the Linux operating system, the Apache web server, and the popular web browser Firefox. OSS has become so widespread and popular that even corporations are increasingly accepting it as a reliable alternative to proprietary software. Over the last few years most large companies in North America have become comfortable with the idea of implementing OSS, even for mission-critical applications (Schadler, 2004).

Some important software makers such as IBM, Sun Microsystems and, to a lesser extent, Microsoft, have jumped on the OSS bandwagon to take advantage of volunteer developer communities for their own projects. However, software firms have not yet tapped the full potential of OSS, nor do we understand how the OSS model can best be leveraged.

A key trait of OSS is its organization. The way OSS developers share work is not easily categorized; their organizational structure is best described as a loose collaboration network (Crowston and Howison, 2005). These networks can adopt a myriad of different configurations, which are closely related to the task allocations of team

members. In fact, when OSS project members work jointly on a task, such as creating a new file or program module, a collaboration tie is generated.

The collection of all collaboration ties present within a team creates an overall pattern of collaboration, or “collaboration structure”, that can be characterized based on network-based parameters such as the strength of the ties or the degree of centralization within that structure (Wasserman and Faust, 1994). In this paper, then, collaboration structure refers to the collection of links and nodes where nodes are developers and there is a tie between any two developers only when those two developers collaborated on a file. The strength of collaboration is given by the number of files they collaborated on. Also, within the collaboration structure, isolates are those developers who do not collaborate with any other developers. Clearly, this collaboration structure can be characterized in terms of network analysis, and be assimilated to a collaboration network.

Then, we can state the main research question of this paper: How do different network-like structures associate with project performance, in terms of project productivity and product quality? The network-based structural characterization of collaboration is well established in the sociological tradition (Wasserman and Faust, 1994), but the approach is not common in the new product development (NPD) literature. Structural collaboration characteristics are obviously not captured by prevalent aggregated measures such as intensity or frequency of collaboration, yet — according to findings from team dynamics — those characteristics can be instrumental in explaining the importance of collaboration for NPD performance (Ahuja *et al.*, 2003). It is important to disentangle which of these structures promote better development performance because a team leader or manager can actively configure their team’s collaboration structure to achieve superior results.

This paper examines relevant hypotheses through a field study of working software open source NPD teams, using archival data from electronic sources. Following the introduction, Section 2 of this paper reviews relevant literature and lays out the hypotheses. Section 3 describes the analytical methods, including the chosen research framework, sampling, measurement, and analysis. Section 4 discusses the results and Section 5 comprises conclusions, limitations, and future research opportunities. The Appendix contains the robustness tests carried out to ascertain the sensitivity of the results to departures from analytical assumptions.

Construct Definition, Background, and Hypotheses

Software development performance

The process of new software development is adequately informed by the general literature on new product development/product innovation. This literature suggests

several suitable metrics for software development performance, but development productivity is particularly recommended (Carrillo, 2005). Productivity and product quality are also well recognized metrics for development performance that were used countless times in the software engineering realm (Jones, 1986). Another example is that of Krishnan *et al.* (2000) who used development productivity and self-reported product quality to measure NPD performance for software products. Other more esoteric metrics have been used in particular cases: Voss (1985) studied a commercial system in its deployment phase and classified success metrics into installation success, commercial success in terms of product use (“user base”), and an ad-hoc composite success measure based on innovation diffusion. For this study the performance metrics used are, inline with most extant literature, product quality and development productivity.

Collaboration structure

We define collaboration as joint work on an intellectually demanding job. The organizational structure of NPD teams (i.e., the collaboration structure studied in this paper) is especially interesting for practitioners and scholars (Brown and Eisenhardt, 1995). It is not only a potential performance correlate, but also a managerially actionable critical decision (Krishnan and Ulrich, 2001). But in order to discover which collaboration structure is best, we first need a way to discriminate among different structures. Social network analysis (SNA) (Wasserman and Faust, 1994) is helpful for that purpose and is adopted in this study. SNA can be used to accurately describe any kind of social structure, based on key parameters derived from graph theory.

A collaboration network is defined by a joint set of actors and ties. The actors in this case are the NPD team members. Two team members share a tie when they collaborate on at least one task (i.e., a software program file).¹ This network is undirected in that there is no net directionality in the exchange between actors. Also, each tie has a certain strength or value, assigned according to the number of tasks worked in common by the corresponding pair of team members.² Several parameters are used to measure conceptually important characteristics of collaboration structures; in this paper the measures used are density, centralization, and boundary spanning activity.

¹Results are consistent if task is defined as software module instead of at the file level.

²Results are consistent if tie strength is dichotomized at their mean or at their median values.

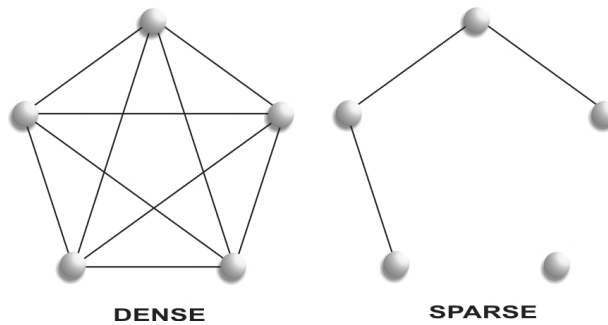


Fig. 1. Density.

Density

An important characteristic of a collaboration structure is the ratio of the number of ties present to the maximum number of ties theoretically possible for a given team size. This is captured by a structural parameter known as density (Wasserman and Faust, 1994).

Density measures the interconnectedness or cohesiveness of a structure. In valued networks (where each tie has a value or “strength”), density is the average value of tie strength standardized by the size of the network (Wasserman and Faust, 1994). Figure 1 compares sparse and dense collaboration networks.

Some researchers have found a positive association between network density and team performance in new product development (Reagans and Zuckerman, 2001). Others argue that the relationship is inverse and U-shaped (Oh *et al.*, 2004; Chidambaran and Tung, 2005). For large teams and relatively complex tasks a negative association between density and performance was found (Balkundi and Harrison, 2006). Although there are few empirical studies specific to software development, the complexity of the task suggests that we should observe a negative relation, at least for relatively large teams (Levina, 2005).

At low network density, team members are sparsely connected and, on average, each member works on a relatively small number of tasks. However, at high density, “everyone is working on everything”. The team members of lower density networks tend to be specialists, but members of higher density networks tend to be generalists who tackle any development task regardless of whether it matches their particular abilities or talents.

Relatively complex software development projects commonly employ specialists for different stages of the project. Studies repeatedly have observed that specialists produce higher quality code than generalists; for instance, in the areas of user interfaces, database management, and software security projects (e.g., Perlman, 1995). Hence, a negative relation can be expected between collaboration

network density and product quality:

H1a: Density is negatively associated with product quality

The software development process involves active information transmission among developers (Levina, 2005), and it is reasonable to tap into elements of information theory (Ash, 1966) to support the hypotheses. Analyzing productivity within an information network can be achieved by examining the effort required to transmit information through network nodes.

Empirically, the relationship between density and productivity has already been observed in an R&D setting, different from this study in that it used perceptual productivity measures and formal corporate teams (Reagans and Zuckerman, 2001). This work revealed that network density in a communication network is related to the mean level of communication throughout the network.

Teams with a high level of communication have access to more resources to accomplish their tasks so they expend less effort finding and using information (such as browsing the Internet or looking for technical manuals). Also, denser teams have fewer coordination problems and tend not to duplicate their work. Dense teams also use direct ties to transmit information — direct ties require less effort to convey information, resulting in higher productivity.

If we reverse the previously assumed causal direction we can expect that there will be more tasks underway in a more productive project, as network density can be interpreted as the average number of tasks carried out in the team by any team member dyad. Both of these arguments suggest that:

H1b: Density is positively associated with development productivity

Centralization

Centralization is another parameter used to characterize the relative importance of different actors in a network. It describes the extent to which the various actors in a network are equally important (i.e., central) or, conversely, the extent to which a few individuals are more important than the rest. In a sense, it is a measure of individual importance (centrality) dispersion and takes the value of zero when all actors are equally important and a higher value when few or even only one actor dominates the other actors in the network (Wasserman and Faust, 1994).

In a centralized structure, one or a few actors hold proportionally more ties than the rest (Wasserman and Faust, 1994). Figure 2 shows an example of two networks, one centralized and the other decentralized. Note that these two networks have the same density.

Centralized structures hinder cooperation (Molm, 1994) because actors need to use indirect paths, mostly through a few central actors that perform as information

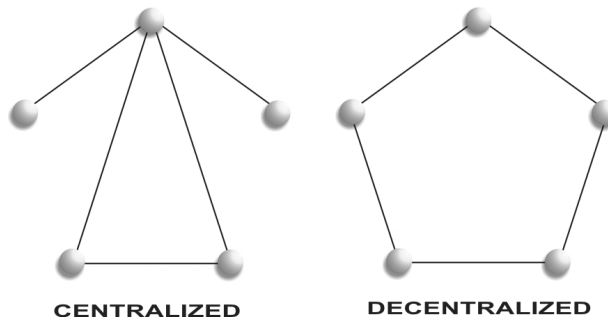


Fig. 2. Centralization.

brokers. Using indirect paths increases the risk of information attenuation or distortion, and distorted information can cause mistakes in execution.

Also, a centralized structure obfuscates direct cooperation between actors, thereby decreasing the quantity of non-redundant knowledge available, something which is related to poorer quality (Levitt and March, 1988; Leonard-Barton, 1992; Teigland and Wasko, 2003). Therefore, we expect that the more centralized the structure, the lower the quality of the resultant product, as follows:

H2a: Centralization is negatively associated with product quality

Extant empirical studies have found weak support for a negative relationship between centralization and self-reported group performance in organizations (Sparrowe *et al.*, 2001). Early studies found that decentralized communication is more efficient for intellectually demanding tasks (Shaw, 1964). NPD is indeed an intellectually demanding activity (Levina, 2005) and one would therefore expect that decentralized teams would be more productive.

Also, software NPD collaborations require team members to share substantial amounts of information (Levina, 2005). In a centralized structure, this means using indirect paths through the more central actors. The total effort needed to transmit information in a centralized structure will therefore be greater than the effort required in a decentralized structure in which there are more direct connections between pairs of nodes. If greater effort is required to transmit information, the centralized structure will be less productive. Then:

H2b: Centralization is negatively associated with development productivity

Boundary spanning activity (BSA)

Early studies on team performance determined that the innovation process not only involves intra-team collaboration, but also collaboration with external entities and other projects (Allen and Cohen, 1969; Allen, 1970; Mallick and Schroeder,

2005). NPD project team members regularly have to cross team boundaries to seek input about market conditions, environmental changes, problem-solving tools and techniques, and so on (Miller, 1971; Utterback, 1974).

Communication that crosses organizational boundaries (in this case the team's) has been called boundary spanning activity (BSA) (Allen, 1970). Previous research has documented the importance of BSA for collecting relevant external information, and there is some empirical support for the argument that BSA improves performance (Allen, 1970; Dollinger, 1984; Brass *et al.*, 2004).

Product innovation is a knowledge-intensive task, yet there can be a great deal of overlap in knowledge available within organizational boundaries. Indeed, the aggregate knowledge of the team may not be much different from what any one member knows (Leonard-Barton, 1992). Further, innovation often occurs under conditions of environmental uncertainty. The knowledge required to perform development tasks can change rapidly and not be available from existing team members. In such a situation, knowledge must be sought from outside of the team. A lack of BSA may then translate into a knowledge gap between what is known and what the team needs to know in order to cope effectively with changes in the environment (Levitt and March, 1988; Leonard-Barton, 1992; Teigland and Wasko, 2003).

Moreover, teams that are isolated from their context can become mired in "groupthink", developing a homogeneous set of beliefs that hinders creativity and penalizes critical analysis (Janis, 1972). "Groupthinkers" have been found to be reluctant to consider external signals demanding change, eventually arriving at decisions of lower quality (Esser, 1988).

Conversely, collecting knowledge that lies beyond formal organizational boundaries can provide essential information and improve organizational capabilities, particularly in terms of innovation performance. With adequate external information, groups can access new ideas and technologies that can be transferred to their tasks in the focal team, thereby improving the quality of their work. Similarly, communicating with other teams can help team members gather information about competitive products, stay ahead of technical developments implemented elsewhere, and incorporate new features requested by users of similar products. Thus, the following hypothesis can be stated:

H3a: BSA is positively associated with product quality

However, maintaining external links is costly in terms of time and effort, especially because knowledge gathered from other projects needs to be reciprocated in order to sustain links (Lerner and Tirole, 2002). Maintaining too many boundary roles rapidly becomes marginally inefficient (Tushman, 1977).

A high level of BSA can place an additional overhead on the individuals who perform boundary spanning roles, particularly in terms of time spent on sense-making and decoding information. In addition, a high degree of BSA can produce information overload, requiring more effort to adapt information from external to internal meaning frameworks. This heightens the risks of bias and distortion, eventually increases the need for rework, and ultimately makes work less efficient.

The competing demands of multiple projects adds to team member stress and intensifies their sense of ambiguity about their role in the focal project (Aldrich and Herker, 1977). This results in decreased job satisfaction, which is associated with lower productivity (Lysonski, 1985). The following hypothesis can therefore be stated:

H3b: BSA is negatively associated with development productivity

There are other variables that may affect NPD team performance. One of the most frequently mentioned is team size, measured by the number of team members. However, the literature is inconclusive about the direction of the association (Damanpour, 1996; Martins *et al.*, 2004). This study takes team size as a control variable, not hypothesizing its relation with performance, but rather exploring it empirically.

Task complexity also influences the team's organization (Baldwin and Clark, 1999) and impacts group performance, although again, the current status of the literature suggests that further empirical studies are needed to confirm the nature of this effect (Guzzo and Dickson, 1996; Martins *et al.*, 2004).

In general, empirical studies on NPD performance also need to control for product complexity (McDermott, 1999). Basili and Hutchens (1983) define software complexity as a measure of the resources expended while interacting with software code, and this complexity arises from the organization of programming elements within the software code. When software is more complex, more information needs to be processed during coding by developers. When they collaborate on more complex software, developers must not only comprehend each of the individual modules, but also spend additional cognitive effort to understand how different modules and paths are interrelated, making collaboration more demanding. This challenge may be translated into an increased effort to fix bugs and a prolonged development speed. A software artifact is more complex when it contains more interrelated modules or paths (Darcy *et al.*, 2005). A widely accepted measure for software complexity is McCabe's cyclomatic complexity, which measures the intricacy of the logic flow in the program (McCabe, 1976). That measure was used in this paper.

Product popularity was also included as a control, measured by the number of times the program files were downloaded by users (Stewart, 2004), since it is argued that the user base influences performance in OSS (Lerner and Tirole, 2002).

Finally, OSS projects work under a handful of different types of software licenses, and license types have been found to motivate developers and be related to the productivity of such projects (Colazo *et al.*, 2005). The resultant research model is depicted in Fig. 1.

Methods

The public availability of data on OSS — including the product (source code) itself and all of the work records in artifacts such as files and modules — represents a great opportunity for empirical research, such as this study.

Sampling

The unit of analysis was the NPD project. OSS project information is hosted in web-based repositories which have been used repeatedly as sources of archival data for empirical studies (Mockus *et al.*, 2002). Similarly, this paper uses Source Forge (SF) (www.sourceforge.net) which is the paramount OSS project repository.

The software projects selected were those developed in the programming language “C”, by six or more team members. This minimum number of team members was chosen to avoid any bias from the large number of one-person projects in SF, while still maintaining a reasonable variability range in the network-based measures. We also chose this minimum because we wanted to study relatively large teams, which are potentially representative of most applications of interest for businesses.

There were several reasons for restricting the study to one programming language. First, a single language is strongly preferred for code-based metrics (Jones, 1986). Second, C is still heavily used, even though it is not a recent programming language. Third, C is particularly suitable for analysis because there are several “tried and true”, off-the-shelf tools that allow researchers to calculate different static metrics such as complexity or productivity. These metrics are well recognized and frequently used by researchers, especially in the computer science field. In the SF repository, there were 276 software projects developed from scratch by six or more team members using C as the sole programming language. These projects constituted the sampling frame for this paper.

The final sample was reduced to 101 projects since only that number had archived full data on all development activities. For the 101 projects, repeated measures were taken on a quarterly basis since the beginning of the project until the date of measurement, December 31st 2005, constituting a repeated measures (panel data) set. Since not all the projects had the same starting date, the number of quarters of data was unequal, with an average of 9 quarters, for a total of 897 data

points. A holdout sample was taken for lineality checks before running the OLS regression model (see the analysis section), and therefore the regressions were run with $N = 807$ data points.

Ordinary least squares (OLS) regression is generally avoided when dealing with panel data, but nothing precludes its application, provided the assumptions of the OLS model are met. Potential issues related to the use of OLS on the present panel data are reviewed later in this section and an Appendix with all the tests performed on the data is available on request. In brief, all the checks performed provided evidence of the validity of OLS results for the present analyses.

Data Collection and Measurement

The data collection strategy is depicted in Fig. 3. The projects' source code files were downloaded and parsed with custom-written software analysis programs to obtain product size in source lines of code (SLOC) as well as other static metrics such as code complexity and number of lines added for each time period, based on standard analytic algorithms (McCabe, 1976). All of these basic metrics were further confirmed using at least two different off-the-shelf code analyzers.

As noted before, development performance was measured using product quality and development productivity. Collaboration structure was described in terms of team size, density, centralization, and boundary spanning activity.

Development productivity was measured using a well established empirical metric for software development productivity: average source lines of code produced per team member, per unit of time (Fenton and Neil, 1999).

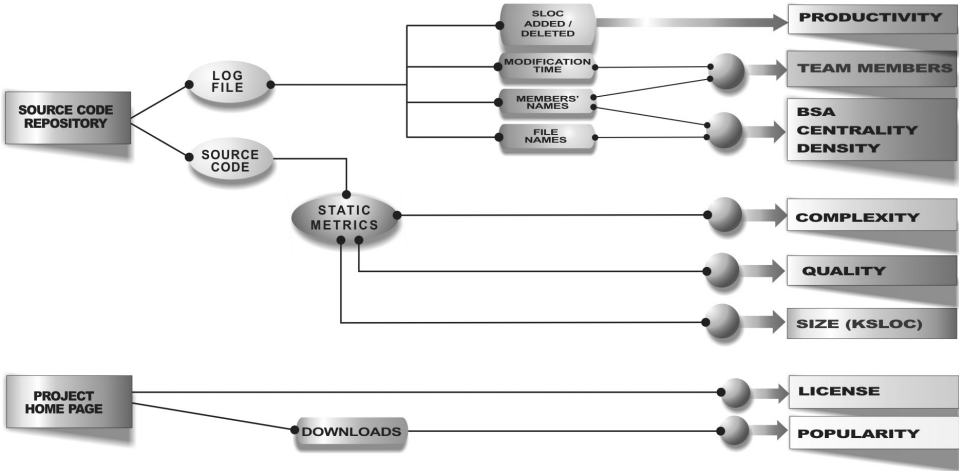


Fig. 3. Data collection.

Product quality was measured by the number of expected software defects “B”, standardized per thousand lines of source code, obtained from the source code files for each project. This metric was pioneered by Ottenstein (1981). The parameter B estimates the expected number of defects latent in the source code and has been repeatedly empirically validated (cf. Gremillion, 1984) against post-test (actually found) quality defects. **For the interpretation of the results, note that a higher defect count corresponds to lower software quality.** This parameter (“B”) was calculated by parsing the source code of each project with an off-the-shelf program commonly used in software development for obtaining static code metrics.

Collaboration structure was measured using social network analysis (Wasserman and Faust, 1994). As mentioned, the collaboration structure is characterized by two main parameters: density and centralization.

A graph is defined as a collection of nodes and arcs linking them. In a graph, density (Δ) is defined as the ratio of the number of arcs to the maximum possible number of arcs. In a valued graph (one in which arcs have a value or “strength”) this concept is extended by assigning to each arc its value and to each missing arc the value of 0. Let g be the number of nodes in a graph, k be the number of arcs, and v_k be the value assigned to arc k . Then:

$$\Delta = \sum v_k / g(g - 1)$$

This measure is exactly the average strength of the arcs in a directional network in which the maximum number of arcs is $g(g - 1)$ (excluding self-loops) and proportional to the average strength of the arcs in a non-directional network in which the maximum number of arcs is $g(g - 1)/2$. For instance for the 5-node non-directional and valued network represented in Fig. 5.

$$\Delta = \frac{3 + 2}{5 * 4} = 0.25$$

There are several measures of the centralization of a social structure, but this paper uses the most common variant of centralization: the one based on the individual nodes’ degree centrality (Freeman *et al.*, 1989). Degree centrality is based on the number of ties actors have. An actor with a large degree centrality is directly connected to a large number of other actors. In this study, this would mean that a team member collaborated with a large number of other members. In a team with greater centralization, a few team members have more ties than the rest. In a team with less centralization, most team members have approximately the same number of ties.

Let $C_D(n_i) = d(n_i)$ be the actor degree centrality for actor i , and let x_{ij} be the value of the arc linking actors i and j (for non-valued networks $x_{ij} = 0$ if actors i and j are not linked and 1 otherwise, and for valued networks $x_{ij} = v_{ij}$; the value

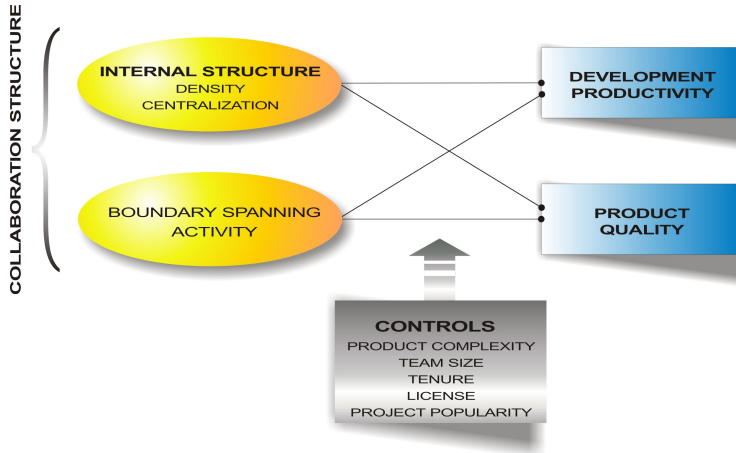


Fig. 4. Research model.

assigned to each arc). Then

$$C_D(n_i) = d(n_i) = \sum_j x_{ij}$$

If all actors in a network had the same degree centrality, then the network itself would be perfectly decentralized, and the distribution of degree centralities in that network would have zero variance. In fact, the concept of degree centrality variance is what defines degree centralization. The greater the degree centrality variance, the more centralized is the network. In the case of valued networks, the variance of the actor-level degree centralities is used.

$$S_D^2 = \left[\sum_{i=1}^{i=g} (C_D(n_i) - \overline{C_D})^2 \right] / g$$

where $\overline{C_D}$ stands for the mean actor degree index. This measure was recommended as a group-level index of centrality by [Snijders \(1981\)](#), reflecting the view that centralization is synonymous with the dispersion or heterogeneity of the actors' indexes ([Hoivik and Gleditsch, 1975](#)). For example, for the graph of Fig. 5.

$$\begin{aligned} g &= 5 \\ \overline{C_D} &= \frac{3 + 2 + 5 + 0 + 0}{5} = 2 \\ S_D^2 &= \frac{[(3 - 2)^2 + (2 - 2)^2 + (5 - 2)^2 + (0 - 2)^2 + (0 - 2)^2]}{5} \\ &= \frac{1 + 0 + 9 + 4 + 4}{5} = \frac{18}{5} = 3.6 \end{aligned}$$

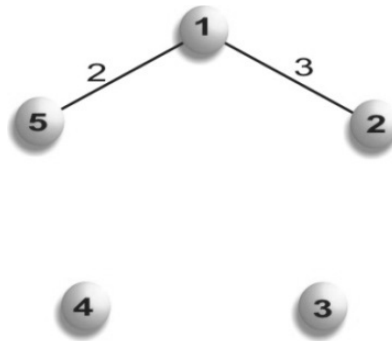


Fig. 5. Density, example.

The network data was obtained from the activity logs of each project's concurrent versioning system (CVS). The CVS is a standard tool that registers each modification a team member makes to any file belonging to the project; it is used to prevent programming conflicts in multi-member projects. The CVS log files contain, among other information, the name of the file modified, the name of the team member making the modification, the kind of modification made, the number of lines added, lines deleted, and lines modified.

A custom-made script downloaded these logs and extracted the detail of who worked on each file. The number of team members was calculated from the CVS logs as the number of different individuals who made changes to the project's files. To avoid collinearity problems with other variables, the number of team members was normalized by dividing it by the product size (log-transformed), which was measured in thousand source lines of code (KSLOC).

Another script translated the CVS log information into associative data relating each possible team member dyad to the files they worked on in common, if any. These network data were fed to "R" (R Development Team, 2006) to obtain the structural parameters (network density and network centralization).

Previous researchers have measured BSA by estimating the number of communication instances between a focal group and other groups (Rosenkopf and Tushman, 1998), or the percentage of time that a given individual dedicates to communicating outside the team, as opposed to communicating internally (Tushman and Scanlan, 1981). These metrics are designed to capture either BSA intensity or BSA frequency. Dollinger (1984) measured both BSA intensity and BSA frequency and found that the two measures were similarly correlated with organizational performance.

SNA offers an elegant alternative for measuring BSA. Networks can be multi-level (i.e., actors form networks that are in turn actors of larger networks) (Wasserman and

Faust, 1994). In this case, one can visualize a two-level arrangement in which a network of members forms a development team at the bottom level and, at a higher level, a network of teams maintains a tie when they share one or more team members. It has been observed that team members in OSS projects often work on more than one project simultaneously (López-Fernández *et al.*, 2004).

Thus, a plausible proxy for a team's BSA is the number of its team members who also work on some other project, standardized by the number of team members of the focal team. To collect this metric, a script detected which team members worked on more than one project and calculated BSA for each project in the sample for each time period.

Controls

The sampling design controlled for programming language to eliminate potential inconsistencies in the metrics based on source code. Project tenure was measured by the number of days between the first input to the source code repository and the time that measurements were taken. Project size was measured in KSLLOC. As mentioned before, task complexity was measured using McCabe's cyclomatic complexity measure (McCabe, 1976). A dummy variable was introduced to distinguish between projects under two main types of licenses: copylefted (roughly, non-appropriable by third parties) and non-copylefted (cf. Dahlander, 2005).

Table 1 summarizes the definition of the constructs, their measures, and the corresponding references that previously suggested or used those measures. The measures were subjected to a content validity assessment (Hinkin, 1998) using a panel of academic software development experts from a university's computer science department. Ten experts were selected based on their concrete experience in developing OSS, provided with construct definitions and asked to match constructs with measures. They came to 90 percent agreement, which is higher than the 75 percent normally indicative of content adequacy (Schriesheim *et al.*, 1993). The hypotheses were tested using OLS regression and time series methods.

Results

A random 10 percent of the data was held out to check for linearity. As a consequence of this test, logarithmic (base 10) transformations were selected for all variables except project tenure, for which an inverse transformation was selected. Further analyses proceeded with only the remaining 90 percent of the sample ($N = 807$), to avoid concerns of sample-induced associations.

Table 2 shows bivariate correlations and basic descriptive statistics. Table 3 shows OLS results with the number of defects per thousand lines of code and

Table 1. Measures.

Construct	Definition	Measures	Reference
BSA	Communication activity that crosses the project’s boundaries	The number of team members in common between the project and other projects, divided by team size	—
License	The presence in the software license of a “copyleft” clause that requires derivative products be released under the same license under which it was acquired	Dummy variable (0 = Non-Copylefted, 1 = Copylefted)	(Colazo <i>et al.</i> , 2005)
Centralization	The extent to which team members differ in importance	Degree centralization for valued networks	(Wasserman and Faust, 1994)
Team Size	Number of team members	Number of programmers	—
Density	The degree of connectedness in the team	Network density for valued networks	(Wasserman and Faust, 1994)
Productivity	Project-average team member productivity	Lines of code per team member per time period	(Fenton and Neil, 1999)
Product Complexity	Logic intricacy, understandability	Average File-Level McCabe’s Cyclomatic Complexity Index	(McCabe, 1976)
Product Quality	Reliability	Expected number of pre-test bugs per thousand lines of source code	(Ottenstein, 1981)
Product Popularity	Usage	Number of product downloads	(Stewart, 2004)
Product Size	The size of the source code base of the project	KSLOC	(Jones, 1986)
Team Tenure	The extent of time the project has been active	The elapsed number of time periods from the beginning of the project	—

Table 2. Bivariate correlations.

	Productivity	B/KSLOC	Popularity	License	Tenure	Complexity	Team Size	Density	Centralization	BSA
Productivity	1									
B/KSLOC	0.080*	1								
Popularity	0.204**	0.170**	1							
License	0.042	0.168**	0.108**	1						
Tenure	0.017	-0.009	0.338**	-0.025	1					
Complexity	0.108**	0.071*	0.040	-0.038	-0.030	1				
Team Size	-0.239**	0.284**	-0.160**	0.120**	-0.295**	-0.078*	1			
Density	0.535**	0.111**	0.089*	-0.020	-0.009	0.070*	-0.128**	1		
Centralization	0.408**	0.017	0.175**	-0.050	-0.081*	0.070*	0.021	0.511**	1	
BSA	-0.030	-0.027	0.178**	0.021	0.089*	-0.103**	0.081*	-0.007	0.135*	1
Mean	0.228	1.802	2.752	0.790	-0.248	0.9732	1.260	0.635	1.402	1.044
Std Dev.	0.762	0.204	1.328	0.410	0.270	0.301	0.255	0.452	1.521	0.277

Notes: N = 807; * p < 0.05; ** p < 0.01.

Table 3. OLS Models.

	Defects/KSLOC Coeff.	Productivity Coeff.
(Constant)	1.45****	− 0.24
Team Size	0.15****	− 0.3****
BSA	− 0.05**	− 0.14*
Centralization	− 0.02***	0.09****
Density	0.09****	0.68****
License	0.05***	0.13**
Tenure	0.02	− 0.15*
Complexity	0.06**	0.11
Popularity	0.03****	0.07****
Coefficients are unstandardized		
N = 807		
F = 22**** F = 58****		
d = 1.97 d = 2.1		
R ² = 0.18 R ² = 0.37		

Notes: ****p < 0.001; ***p < 0.01; **p < 0.05; *p < 0.1.

productivity (note that a positive regression coefficient represents a negative association with quality). Product size was excluded as a control because of high collinearity.

Discussion and Conclusions

H1a and H1b were supported — density was negatively associated with quality (or equivalently, density was positively associated with number of defects) and positively associated with development productivity.

These findings have direct implications for NPD project managers. If productivity is pursued, all or most team members should be allowed to work on any available task, producing a collaboration network that is dense rather than sparse. However, the negative association with quality creates a difficult trade-off for managers. How should they approach this balancing act? The theory of manufacturing performance frontiers (Schmenner and Swink, 1998; Vastag, 2000) suggests that superior performance in both dimensions of an apparent trade-off situation can be achieved by raising the pertinent asset frontier. We argue that the asset of interest in the trade-off between productivity and quality in relation to collaboration density is the level cross-training of NPD team members. In fact, both the number of tasks that any dyad can attempt and the quality of their

performance can be expected to increase if the members are more knowledgeable about the different types of tasks they might have to carry out.

H2a and H2b were not supported — centralization was positively associated with quality and productivity. These results are unexpected but they consistent with the performance of well-known OSS projects that, at least from a point of view of formal authority, were highly centralized *and* very successful, such as Linux and Apache (c.f. [Crowston and Howison, 2003](#)). Future work should explore whether other definitions of centralization (e.g., betweenness centralization or information centralization) ([Wasserman and Faust, 1994](#)) yield similar results. Of course, another alternative is attempting to confirm these results by applying other theoretical lenses to the problem.

H3a and H3b were both supported — BSA was positively associated with quality but negatively associated with productivity. The results clearly show that in highly productive NPD projects, team members are dedicated to internal activities and not much involved in external collaborations with other projects. But there is a potential caveat: isolating team members from other projects undermines job enrichment and can have a deleterious impact on both motivation and loyalty. However, if quality is preferred rather than productivity, team members should be encouraged to work on a portfolio of different projects so that they can detect and incorporate external knowledge useful to the focal project. But, managers need to think carefully about task allocation in order to lessen potential unwanted side effects.

There are several practical implications of these results. They can be summarized in the form of a to-do list for software NPD project managers:

First: When striving for high quality, let team members participate in multiple projects; when striving for productivity, be wary of the same. The results on the association between BSA and performance show a positive relation with product quality and a negative one with productivity. Allowing software developers to participate in more than one project exposes them to new ideas and improves their problem-solving skills, but the effort spent on those projects undermines their productivity in the focal project.

Second: Make sure there is at least one, but no more than a few, members able to collaborate on a wide variety of tasks with the majority of the other team members. This paper's findings suggest that more centralized collaboration structures yield better quality and productivity. When staffing projects, managers should bear in mind that a more centralized collaboration structure contains a relatively small number of team members who collaborate with most of the other members, and a majority of members that collaborate with few others.

Third: Encourage those team members that collaborate with others to do so in an unrestricted fashion. Findings suggest that increased density is associated

with increased quality and productivity. To foster higher collaboration density in a centralized structure, every collaborating dyad needs to increase its tie value (i.e., the number of tasks on which the member pairs collaborate).

There is also one practical prescription arising from results concerning the team size control variable: Increasing the number of team members for a product of a relatively constant size results in poorer quality and lower development productivity. This finding confirms empirically the anecdotal observation that adding more developers to a late project will often only make it later (Brooks, 1995).

From an academic point of view, this study is the first to consider which collaboration structure is associated with better quality and productivity in the increasingly important computer software industry. From a research design standpoint, this study has several unique and desirable characteristics: it uses “hard”, objective metrics and a large sample of actual, working teams; and it specifies collaboration structure by network-based instead of aggregated measures. Moreover, this paper is the first to simultaneously consider intra- and inter-team collaboration.

Limitations and Future Research

Unfortunately, limitations are unavoidable in empirical research and they should be clearly stated. First, the sample excludes teams of fewer than six members. However, few important software NPD projects are composed of fewer than six people, and it can safely be assumed that this sample represents most relevant projects. Second, the paper relies solely on archival data, which may introduce method bias.

Future research should explore whether the same findings are observed with other empirical methods, such as survey studies. Future research could also investigate the relationship between collaboration structure and other performance metrics, such as team member turnover, development cycle time (Carmel, 1995; Rauscher and Smith, 1995), or flexibility. Another interesting problem is to identify the optimum ratio of team members to product size for different contingencies (e.g., other programming languages, types of application, etc.). It would also be valuable to better understand the characteristics of those team members who are more central in the collaboration network.

References

- Ahuja, MK and DF Galletta *et al.* (2003). Individual centrality and performance in virtual R&D groups: An empirical study. *Management Science*, 49(1), 21–38.
- Aldrich, H and D Herker (1977). Boundary spanning roles and organizational structure. *The Academy of Management Review*, 2(2), 217–230.

- Allen, TJ (1970). Communications networks in R&D laboratories. *R&D Management*, 1(1), 14–21.
- Allen, TJ and S Cohen (1969). Information flow in R&D labs. *Administrative Science Quarterly*, 14(1), 12–19.
- Ash, RB (1966). *Information Theory*. New York, NY: Wiley and Sons.
- Baldwin, CY and KB Clark (1999). *Design Rules*. Cambridge, MA: MIT Press.
- Balkundi, P and DA Harrison (2006). Ties, leaders and time in teams: Strong inference about network structure's effects on team viability and performance. *Academy of Management Journal*, 49(1), 49–68.
- Basili, V and D Hutchens (1983). An empirical study of a syntactic complexity family. *IEEE Transactions on Software Engineering*, 664–672.
- Beck, K (1999). *Extreme Programming Explained: Embrace Change*. Reading, MA: Addison-Wesley.
- Brass, DJ and J Galaskiewicz *et al.* (2004). Taking stock of networks and organizations: A multilevel perspective. *Academy of Management Journal*, 47(6), 795–817.
- Brooks, FP (1995). *The Mythical Man-Month: Essays of Software Engineering (Anniversary Edition)*. Reading, MA: Addison Wesley Professional.
- Brown, SL and KM Eisenhardt (1995). Product development: Past research, present findings and future directions. *Academy of Management Review*, 20(2), 343–378.
- Business Week (2005). Special Report: Get Creative! August 1, 2005.
- Carmel, E (1995). Cycle time in packaged software firms. *Journal of Product Innovation Management*, 12(2), 110–123.
- Carrillo, JE (2005). Industry clockspeed and the pace of new product development. *Production and Operations Management*, 14(2), 125–141.
- Chidambaran, L and LL Tung (2005). Is out of sight, out of mind? An empirical study of social loafing in technology-supported groups. *Information Systems Research*, 16(2), 149–168.
- Colazo, JA and Y Fang *et al.* (2005). *Development Success in Open Source Software Projects: Exploring the Impact of Copylefted Licenses*. Eleventh Americas Conference on Information Systems, Omaha, NE, USA.
- Crowston, K and J Howison (2003). *The Social Structure of Free and Open Source Software Development*. International Conference on Information Systems.
- Crowston, K and J Howison (2005). The social structure of free and open source software development. *First Monday* February 2005.
- Cusumano, M (2004). *The Business of Software*. New York: Free Press.
- Damanpour, F (1996). Organizational complexity and innovation: Developing and testing multiple contingency models. *Management Science*, 42(5), 693–716.
- Darcy, DP and CF Kemerer *et al.* (2005). The structural complexity of software: An experimental test. *IEEE Transactions on Software Engineering*, 31(11), 982–995.
- Dollinger, MJ (1984). Environmental boundary spanning and information processing effects on organizational performance. *The Academy of Management Journal*, 27(2), 351–368.

- Esser, JK (1988). Alive and well after 25 years: A review of groupthink research. *Organizational Behavior and Human Decision Processes*, 73(23), 116–141.
- Fenton, NE and M Neil (1999). Software metrics: Successes, failures and new directions. *The Journal of Systems and Software*, 47(2/3), 149–157.
- Freeman, LC and DR White *et al.*, Eds (1989). *Research Methods in Social Network Analysis*. Fairfax, VA: George Mason University Press.
- Gordon, RJ (2003). Hi-Tech innovation and productivity growth: Does supply create its own demand? *National Bureau of Economic Research Working Papers*. Cambridge, MA: 1–51.
- Gremillion, LL (1984). Determinants of program repair maintenance requirements. *Communications of the ACM*, 27(8), 826–832.
- Guzzo, RA and MW Dickson (1996). Teams in organizations: Recent research on performance and effectiveness. *Annual Review of Psychology*, 47, 307–338.
- Hinkin, TR (1998). A brief tutorial on the development of measures for use in survey questionnaires. *Organizational Research Methods*, 1(1), 104–121.
- Hoivik, T and NP Gleditsch, Eds. (1975). *Structural Parameters of Graphs: A Theoretical Investigation*. Quantitative Sociology. New York: Academic Press.
- Janis, IL (1972). *Victims of Groupthink*. Boston, MA: Houghton Mifflin.
- Jones, C. (1986). *Programming Productivity*. New York: McGraw-Hill.
- Krishnan, MS and KC H *et al.* (2000). An empirical analysis of productivity and quality in software products. *Management Science*, 46(6), 745–759.
- Krishnan, V and KT Ulrich (2001). Product development decisions: A review of the literature. *Management Science*, 47(1), 1–21.
- Leonard-Barton, D (1992). Core capabilities and core rigidities: A paradox in managing new product development. *Strategic Management Journal*, 13 (Special Issue: Strategy Process: Managing Corporate Self-Renewal), 111–125.
- Lerner, J and J Tirole (2002). Some simple economics of open source. *Journal of Industrial Economics*, 50(2), 197–234.
- Levina, N (2005). Collaborating on multiparty information systems development projects: A collective reflection-in-action view. *Information Systems Research*, 16(2), 109–130.
- Levitt, B and JG March (1988). Organizational learning. *Annual Review of Sociology*, 14(8), 319–340.
- López-Fernández, L and G Robles *et al.* (2004). *Applying Social Network Analysis to the Information in CVS Repositories*. International Workshop on Mining Software Repositories, Toronto, Canada.
- Lyonski, S (1985). A boundary theory investigation of product manager's role. *Journal of Marketing*, 49(1), 26–40.
- Mallick, DN and RG Schroeder (2005). An integrated framework for measuring product development performance in high technology industries. *Production and Operations Management*, 14(2), 142.
- Martins, LL and LL Gilson *et al.* (2004). Virtual teams: What do we know and where do we go from here? *Journal of Management*, 30(6), 805–835.

- McCabe, T (1976). A software complexity measure. *IEEE Transactions on Software Engineering*, SE-2 (4), 308–320.
- McDermott, CM (1999). Managing radical product development in large manufacturing firms: A longitudinal study. *Journal of Operations Management*, 17(6), 631–644.
- Miller, R (1971). *Innovation, Organization and Environment*. Sherbrooke, QC, Canada: University of Sherbrooke Press.
- Mockus, A and RT Fielding *et al.* (2002). Two case studies of open source software development: Apache and mozilla. *ACM Transactions on Software Engineering and Methodology*, 11(3), 309–346.
- Molm, LD (1994). Dependence and risk: Transforming and structure of social research. *Social Psychology Quarterly*, 57(3), 163–176.
- Oh, H and MH Chung *et al.* (2004). Group social capital and group effectiveness: The role of informal socializing ties. *Academy of Management Journal*, 47(6), 860–875.
- Ottenstein, L (1981). Predicting number of errors using software science. *ACM SIGMETRICS Performance Evaluation Review*, 10(1), 157–167.
- Perlman, G (1995). *Teaching User Interface Development to Software Developers*. Conference on Human Factors in Computing Systems, Denver, CO, ACM.
- R Development Team (2006). R v2.5.0.
- Rauscher, TG and PG Smith (1995). Time-driven development of software in manufactured goods. *Journal of Product Innovation Management*, 12(3), 186–199.
- Reagans, R and EW Zuckerman (2001). Networks, diversity and productivity: The social capital of corporate R&D teams. *Organization Science*, 12(4), 502–517.
- Rosenkopf, L and ML Tushman (1998). The coevolution of community networks and technology: Lessons from the flight simulation industry. *Industrial and Corporate Change*, 7(2), 311–346.
- Royce, WW (1970). *Managing the Development of Large Software Systems*. IEEE WESCON, reprinted in Proceedings of the 9th International Conference on Software Engineering, Monterey, CA.
- Schadler, T (2004). *Open Source Moves into the Mainstream*. Cambridge, MA, Forrester Research, Inc.: 1–5.
- Schmenner, RW and ML Swink (1998). On theory in operations management. *Journal of Operations Management*, 17(1), 97–113.
- Schriesheim, CA and KJ Powers *et al.* (1993). Improving construct measurement in management research: Comments and a quantitative approach for assessing the theoretical content adequacy of paper-and-pencil survey-type instruments. *Journal of Management*, 19(2), 385–417.
- Shaw, ME (1964). Communication networks. *Advances in Experimental Social Psychology*. L. Berkowitz. New York, Academic Press. 1, 111–147.
- Snijders, T (1981). The degree variance: An index of graph heterogeneity. *Social Networks*, 3(3), 163–174.
- Sparrowe, RT and RT Liden *et al.* (2001). Social networks and the performance of individuals and groups. *Academy of Management Journal*, 44(2), 316–325.

- Stewart, KJ (2004). *OSS Success: From Internal Dynamics to External Impact*. 4th Workshop on Open Source Software Engineering, International Conference on Software Engineering, Edinburgh, UK.
- Teigland, R and MM Wasko (2003). Integrating knowledge through information trading: Examining the relationship between boundary spanning communication and individual performance. *Decision Sciences* 34(2), 261–286.
- Tushman, ML (1977). Special boundary roles in the innovation process. *Administrative Science Quarterly*, 22(4), 587–605.
- Tushman, ML and TJ Scanlan (1981). Boundary spanning individuals: Their role in information transfer and its antecedents. *Academy of Management Journal*, 24(2), 289–305.
- U.S. House of Representatives (2001). Hearing on FAA’s Efforts to Modernize the Air Traffic Control System. *Transportation and Infrastructure*. Washington, D.C.
- Utterback, JM (1974). Innovation in industry and the diffusion of technology. *Science*, 183(4125), 620–626.
- Vastag, G (2000). The theory of performance frontiers. *Journal of Operations Management*, 18(3), 353–360.
- Voss, CA (1985). Determinants of success in the development of applications software. *Journal of Product Innovation Management*, 2(2), 122–129.
- Wasserman, S and K Faust (1994). *Social Network Analysis: Methods and Applications*. Cambridge, U.K: Cambridge University Press.